

运筹学优化算法

线性规划 · 非线性规划 · 凸优化 · 机器学习与大模型应用

运筹学课件

Operations Research & Optimization

2026 年 7 月 31 日

- Part 1 线性规划 (Linear Programming)
- Part 2 非线性规划 (Nonlinear Programming)
- Part 2.5 启发式与元启发式算法 (Heuristics & Metaheuristics)
- Part 3 凸优化 (Convex Optimization)
- Part 4 运筹优化在 ML 与大模型中的应用
- Part 5 GPU / CUDA 并行计算与 ML 优化加速

共计 61 张幻灯片

线性规划简介

线性规划 (LP) 是运筹学中最基础也是最重要的优化模型。

- 目标函数是决策变量的线性函数
- 约束条件是一组线性等式/不等式
- 决策变量可以取连续值（早期经典 LP 中为非负）

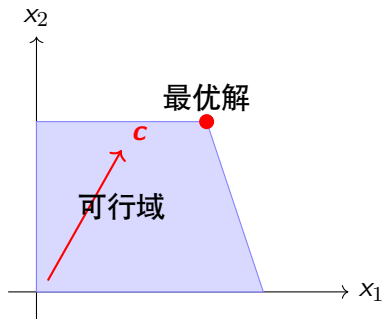
标准形 (Standard Form):

$$\begin{array}{ll}\min_{\mathbf{x}} & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}\end{array}$$

其中 $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$ 。

LP 的几何意义

- 可行域是多面体 (Polyhedron): $\{x \mid Ax = b, x \geq 0\}$
- 若可行域非空且有界, 则为凸多胞形 (Convex Polytope)
- 最优解必在顶点 (extreme point / basic feasible solution) 处取得
- 单纯形法 (Simplex Method) 正是沿着顶点间迭代



LP 的变形

任何 LP 都可以转化为标准形：

- $\max \mathbf{c}^\top \mathbf{x} \iff \min -\mathbf{c}^\top \mathbf{x}$
- $\mathbf{Ax} \leq \mathbf{b} \iff \mathbf{Ax} + \mathbf{s} = \mathbf{b}, \mathbf{s} \geq 0$ (松弛变量)
- $\mathbf{Ax} \geq \mathbf{b} \iff \mathbf{Ax} - \mathbf{s} = \mathbf{b}, \mathbf{s} \geq 0$ (剩余变量)
- x_i 无约束 $\iff x_i = x_i^+ - x_i^-, x_i^+, x_i^- \geq 0$

重要性质

LP 可在多项式时间内求解（椭球法 / 内点法），但 Simplex 法在实践中仍然极其高效。

单纯形法基本思路

基本可行解 (Basic Feasible Solution, BFS): A 的列中选 m 个线性无关列构成基矩阵 B , 令非基变量为 0, 解 $Bx_B = b$ 。

单纯形法迭代步骤:

- ① 从一个初始 BFS 出发
- ② 计算简约代价 (reduced cost): $\bar{c}_N = c_N^\top - c_B^\top B^{-1}N$
- ③ 若 $\bar{c}_N \geq 0$ (极小化问题), 当前解最优
- ④ 否则选入基变量 (Dantzig 规则: 选最负), 出基变量经最小比值 (ratio test) 确定
- ⑤ 更新基矩阵, 进入下一轮迭代

几何上等价于沿着多面体的棱在顶点间移动。

单纯形表 (Simplex Tableau)

标准形 LP 的初始单纯形表:

基变量	x_B	x_N	RHS
x_B	I	$B^{-1}N$	$B^{-1}b$
z	0	$\bar{c}_N^T$	$c_B^T B^{-1}b$

- $\bar{c}_N \geq 0$ 时达到最优
- 若有 $\bar{c}_j < 0$ 且对应列 ≤ 0 , 则问题无界
- Min ratio test 保证可行性

两阶段法 (Two-Phase Method)

初始 BFS 通常不易直接获得。**两阶段法**通过引入人工变量解决：

- **Phase I**: 引入人工变量 $\mathbf{a}$, 求解辅助问题

$$\min \sum_i a_i \quad \text{s.t.} \quad \mathbf{Ax} + \mathbf{a} = \mathbf{b}, \mathbf{x}, \mathbf{a} \geq 0$$

若最优值 > 0 , 原问题无可行解

- **Phase II**: 以 Phase I 的最优 BFS 为起点, 求解原问题
- 大 M 法是其近似替代 (在目标函数中加入惩罚项 $M \sum a_i$)

对偶理论：构造对偶问题

原始问题 (Primal):

$$\min_{\mathbf{x}} \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{Ax} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}$$

对偶问题 (Dual):

$$\max_{\mathbf{y}} \mathbf{b}^\top \mathbf{y} \quad \text{s.t.} \quad \mathbf{A}^\top \mathbf{y} \leq \mathbf{c}$$

Primal (极小化)	Dual (极大化)
变量 $x_i \geq 0$	$\mathbf{a}_i^\top \mathbf{y} \leq c_i$
变量 x_i 无约束	$\mathbf{a}_i^\top \mathbf{y} = c_i$
$\mathbf{a}_i^\top \mathbf{x} \geq b_i$	变量 $y_i \geq 0$
$\mathbf{a}_i^\top \mathbf{x} = b_i$	变量 y_i 无约束

弱对偶与强对偶

弱对偶定理 (Weak Duality)

对于任意可行 $\mathbf{x}$ (Primal) 和 $\mathbf{y}$ (Dual), 有:

$$\mathbf{c}^\top \mathbf{x} \geq \mathbf{b}^\top \mathbf{y}$$

对偶问题最优值给出原始问题最优值的下界。

强对偶定理 (Strong Duality)

若 Primal 有有限最优解, 则 Dual 也有最优解, 且:

$$\min \mathbf{c}^\top \mathbf{x} = \max \mathbf{b}^\top \mathbf{y}$$

互补松弛条件 (Complementary Slackness)

$x_i^* \cdot (\mathbf{a}_i^\top \mathbf{y}^* - c_i) = 0$ 对所有 i ;

$y_i^* \cdot (\mathbf{a}_i^\top \mathbf{x}^* - b_i) = 0$ 对所有 i ;

对偶理论的经济解释：影子价格

- 对偶变量 y 也称影子价格 (Shadow Price)
- $y_i =$ 第 i 个约束右端项 b_i 每增加 1 单位, 最优目标值的边际改善
- 影子价格为 0 的约束是非紧的 (有松弛)
- 影子价格 > 0 的约束是紧的 (资源稀缺)

灵敏度分析 (Sensitivity Analysis) 即利用对偶信息, 判断 b_i 或 c_j 在什么范围内波动后当前基仍然最优。

- 右端项范围: $B^{-1}b \geq 0$ 的范围
- 目标系数范围: $\bar{c}_N \geq 0$ 的范围

线性规划的内点法

LP 也可以用内点法 (Interior Point Method) 求解, 在可行域内部迭代, 多项式时间保证。

原始-对偶路径跟踪法: 解 KKT 系统的扰动形式

$$\begin{aligned} \mathbf{A}\mathbf{x} &= \mathbf{b}, \quad \mathbf{x} > \mathbf{0} \\ \mathbf{A}^\top \mathbf{y} + \mathbf{s} &= \mathbf{c}, \quad \mathbf{s} > \mathbf{0} \\ \mathbf{X}\mathbf{S}\mathbf{e} &= \mu \mathbf{e} \end{aligned}$$

其中 $\mathbf{X} = \text{diag}(\mathbf{x})$, $\mathbf{S} = \text{diag}(\mathbf{s})$, $\mu \rightarrow 0$ 。

用 Newton 法求解该系统, 逐步降低 μ 。实践中对大规模稀疏 LP 效果极好。

- 生产计划：多产品多资源在约束下最大化利润
- 运输问题： m 个产地、 n 个销地的最小成本运输方案
- 分配问题：Hungarian 算法求解 n 对 n 的最优匹配
- 网络流问题：最大流 / 最小费用流 $\rightarrow$ 特殊 LP 结构
- 投资组合：线性约束下的资产配置
- 数据包络分析 (DEA)：效率评价

大规模 LP 求解器：Gurobi, CPLEX, HiGHS, GLPK

非线性规划简介

NLP 的标准形式:

$$\begin{array}{ll}\min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{s.t.} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p\end{array}$$

其中 f, g_i, h_j 至少有一个是非线性的。

- 无约束 NLP: 只需 $\min f(\mathbf{x})$
- 约束 NLP: 需要处理等式和不等式约束
- 解可能非全局最优 $\rightarrow$ 需要全局优化技术

无约束最优性条件

对于 $\min f(\mathbf{x})$, $\mathbf{x} \in \mathbb{R}^n$:

一阶必要条件

若 $\mathbf{x}^*$ 是局部极小点且 f 可微, 则 $\nabla f(\mathbf{x}^*) = \mathbf{0}$ 。

二阶必要条件

若 $\mathbf{x}^*$ 是局部极小点且 f 二阶可微, 则 $\nabla f(\mathbf{x}^*) = \mathbf{0}$ 且 $\nabla^2 f(\mathbf{x}^*) \succcurlyeq \mathbf{0}$ (半正定)。

二阶充分条件

若 $\nabla f(\mathbf{x}^*) = \mathbf{0}$ 且 $\nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$ (严格正定), 则 $\mathbf{x}^*$ 是严格局部极小点。

梯度下降法 (Gradient Descent)

迭代形式:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$$

- α_k : 步长 (step size / learning rate)
- 收敛速度: 线性收敛 (对一般凸函数), $\mathcal{O}(1/k)$ 次梯度收敛
- **Wolfe 条件**确保充分下降和曲率
- **Backtracking line search** 是常用的步长选择策略

缺点: 收敛慢, 对 ill-conditioned 问题表现差, 需精细调整步长。

牛顿法 (Newton's Method)

利用二阶 Taylor 展开：

$$f(\mathbf{x}_k + \mathbf{p}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \nabla^2 f(\mathbf{x}_k) \mathbf{p}$$

Newton 方向： $\mathbf{p}_k = -[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

- 二次终止性：对严格凸二次函数一步到最优
- 局部二次收敛（极快）
- **缺点**：每次迭代需计算和分解 Hessian ($\mathcal{O}(n^3)$)，Hessian 可能非正定

拟牛顿法 (Quasi-Newton Methods)

避免计算 Hessian, 用梯度信息近似 Hessian 逆。

BFGS (Broyden-Fletcher-Goldfarb-Shanno):

$$s_k = \mathbf{x}_{k+1} - \mathbf{x}_k$$

$$y_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$$

$$\mathbf{H}_{k+1} = \left(\mathbf{I} - \rho_k s_k y_k^\top \right) \mathbf{H}_k \left(\mathbf{I} - \rho_k y_k s_k^\top \right) + \rho_k s_k s_k^\top$$

其中 $\rho_k = 1/(y_k^\top s_k)$ 。

- 超线性收敛
- **L-BFGS**: 仅存储最近 m 对 (s_k, y_k) , 内存 $\mathcal{O}(mn)$, 适合大规模问题

Karush–Kuhn–Tucker (KKT) 条件

NLP 的一阶必要条件——KKT 条件:

$$\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \mu_i \nabla g_i(\mathbf{x}^*) + \sum_{j=1}^p \lambda_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}$$

$$g_i(\mathbf{x}^*) \leq 0, \quad i = 1, \dots, m$$

$$h_j(\mathbf{x}^*) = 0, \quad j = 1, \dots, p$$

$$\mu_i \geq 0, \quad i = 1, \dots, m$$

$$\mu_i \cdot g_i(\mathbf{x}^*) = 0, \quad i = 1, \dots, m \quad (\text{互补松弛})$$

- μ_i : 不等式约束的 Lagrange 乘子
- λ_j : 等式约束的 Lagrange 乘子
- 在凸规划中, KKT 条件既是必要也是充分的

约束优化的数值方法

- **罚函数法 (Penalty Method):** 将约束惩罚加入目标

$$\min_{\mathbf{x}} f(\mathbf{x}) + \rho \left[\sum_i \max(0, g_i(\mathbf{x}))^2 + \sum_j h_j(\mathbf{x})^2 \right]$$

$\rho \rightarrow \infty$ 时收敛, 但可能导致 ill-conditioning

- **障碍函数法 (Barrier Method):** 用对数障碍处理不等式约束

$$\min_{\mathbf{x}} f(\mathbf{x}) - \mu \sum_i \ln(-g_i(\mathbf{x}))$$

$\mu \rightarrow 0$ 时收敛

- **SQP (Sequential Quadratic Programming):** 每步解一个二次规划 (QP) 子问题
- **增广 Lagrange 法 (Augmented Lagrangian):** 结合罚函数和对偶思想

信赖域方法 (Trust Region Methods)

每步在一个信赖域半径 Δ_k 内近似目标函数:

$$\min_{\|\mathbf{p}\| \leq \Delta_k} m_k(\mathbf{p}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \mathbf{B}_k \mathbf{p}$$

根据实际下降与模型预测下降的比率 ρ_k 决定是否接受步及如何调整 Δ_k :

$$\rho_k = \frac{f(\mathbf{x}_k) - f(\mathbf{x}_k + \mathbf{p}_k)}{m_k(\mathbf{0}) - m_k(\mathbf{p}_k)}$$

- 比线搜索更稳健，能自动调节步长
- 适合非凸和 ill-conditioned 问题
- 在大规模机器学习中有应用（信赖域 Newton 法）

为什么需要启发式算法？

NLP 中许多实际问题是非凸、NP-hard 的，精确算法在可接受时间内无法求解。

启发式 (Heuristic): 基于问题特定知识的经验规则，能快速找到较好解，但不保证最优。

元启发式 (Metaheuristic): 问题无关的高层搜索策略，指导底层启发式探索解空间。

核心思想

在 exploitation (利用) 和 exploration (探索) 之间平衡，避免陷入局部最优。

单点搜索:

- 局部搜索 / 爬山法
- 模拟退火 (SA)
- 禁忌搜索 (TS)
- 变邻域搜索 (VNS)

群体搜索:

- 遗传算法 (GA)
- 粒子群优化 (PSO)
- 蚁群算法 (ACO)
- 差分进化 (DE)

贪心算法与局部搜索

贪心算法 (Greedy): 每一步选择当前最优, 不回溯。速度快但易入局部最优。

局部搜索 (Local Search) / 爬山法 (Hill Climbing):

- 从初始解出发, 在邻域 $N(x)$ 中迭代改进
- 若 $f(x') < f(x)$, 接受 x' , 否则停止
- 典型结果: 局部最优——无法通过邻域移动找到更好解

困境

纯局部搜索在复杂的“崎岖”解空间中极易卡在低质量局部最优。元启发式通过接受“坏”移动来逃脱局部最优。

模拟退火 (SA)

模拟金属退火的物理过程：高温 $\rightarrow$ 原子活跃跳跃 $\rightarrow$ 缓慢冷却 $\rightarrow$ 结晶到最低能态。

Metropolis 准则（概率接受劣解）：

- 若 $\Delta E = f(\mathbf{x}_{\text{new}}) - f(\mathbf{x}) < 0$ ：接受新解
- 若 $\Delta E \geq 0$ ：以概率 $p = \exp(-\Delta E / T)$ 接受

温度 T 从高逐渐降低（冷却调度 cooling schedule），早期 T 高时几乎接受所有移动。

关键参数：

- 初始温度 T_0 ：需足够高
- 冷却率 α ： $T_{k+1} = \alpha \cdot T_k$, $\alpha \in (0.9, 0.99)$
- 终止条件： $T < T_{\min}$ 或最大迭代次数

理论上以概率 1 收敛到全局最优（当冷却足够缓慢），但实践中需要合理调参。

遗传算法 (GA)

模拟自然选择和遗传进化过程。

基本流程：

- ① 初始化：随机生成种群 (population)
- ② 适应度评估：计算每个个体的 $f(x)$
- ③ 选择 (Selection)：轮盘赌 / 锦标赛选择——优者更可能被选中
- ④ 交叉 (Crossover)：两个父代交换基因片段产生子代
- ⑤ 变异 (Mutation)：以低概率随机扰动基因，维持多样性
- ⑥ 替换：新一代取代旧一代，重复 2-5

编码方式：二进制编码、实数编码、排列编码 (TSP 等组合问题)。

- 交叉率 $p_c \approx 0.7-0.9$ ，变异率 $p_m \approx 0.001-0.1$
- 精英策略 (Elitism)：保留最优个体直接进入下一代

粒子群优化 (PSO)

模拟鸟群、鱼群的群体协作行为。

每个粒子 i 在搜索空间中飞行，记录：

- $\mathbf{x}_i$: 当前位置
- $\mathbf{v}_i$: 速度
- $\mathbf{p}_{\text{best},i}$: 个人历史最优位置
- $\mathbf{g}_{\text{best}}$: 全局最优位置

速度与位置更新：

$$\mathbf{v}_i \leftarrow \omega \mathbf{v}_i + c_1 r_1 (\mathbf{p}_{\text{best},i} - \mathbf{x}_i) + c_2 r_2 (\mathbf{g}_{\text{best}} - \mathbf{x}_i)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- ω : 惯性权重 (控制探索 vs 利用)
- c_1, c_2 : 认知 / 社会学习因子 (典型值 ≈ 2)
- 实数连续优化问题的主力算法之一

禁忌搜索 (Tabu Search)

通过禁忌列表 (Tabu List) 记忆近期访问过的解或操作，强制探索新区域。

核心机制：

- 即使邻域中没有更好的解，也移动到最优的“非禁忌”邻居
- 禁忌长度 (Tabu Tenure)：记录最近 ℓ 步禁止的移动
- 特赦准则 (Aspiration Criterion)：若禁忌移动能产生历史最佳解，可破例接受
- 长期记忆 (Long-term Memory)：强化好区域、惩罚探索过的区域（频率记忆）

特别擅长组合优化问题：TSP、调度、图着色等。

蚁群算法 (Ant Colony Optimization, ACO)

模拟蚂蚁觅食的信息素机制：蚂蚁在走过的路径上释放信息素 (pheromone)，后续蚂蚁倾向于选择信息素浓度高的路径。

TSP 上的 ACO:

- 信息素 τ_{ij} 在边 (i, j) 上积累
- 蚂蚁 k 从城市 i 选择下一城市 j 的概率:

$$p_{ij}^k = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{\ell \in N_i^k} \tau_{i\ell}^{\alpha} \cdot \eta_{i\ell}^{\beta}}$$

其中 $\eta_{ij} = 1/d_{ij}$ (能见度 / 启发式信息)

- 信息素蒸发 (Evaporation): $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$, 避免过早收敛
- 精英蚂蚁加强最短路径上的信息素

广泛应用于路径规划、网络路由、调度等组合优化问题。

启发式算法对比

算法	搜索模式	记忆	典型问题
SA	单点 + 概率跳跃	无	连续 / 组合
GA	群体 + 交叉变异	隐性 (种群)	组合 / 连续
PSO	群体 + 速度驱动	p_{best}, g_{best}	连续
TS	单点 + 禁忌引导	显性 (列表)	组合
ACO	群体 + 信息素	分布式 (信息素)	路径 / 调度

No Free Lunch Theorem: 没有一个算法在所有问题上都最优。选择算法取决于问题结构、维度、约束类型。

凸集 (Convex Sets)

集合 $C \subseteq \mathbb{R}^n$ 为凸集, 若对任意 $\mathbf{x}, \mathbf{y} \in C$ 和 $\theta \in [0, 1]$:

$$\theta \mathbf{x} + (1 - \theta) \mathbf{y} \in C$$

常见凸集:

- 超平面: $\{\mathbf{x} \mid \mathbf{a}^\top \mathbf{x} = b\}$
- 半空间: $\{\mathbf{x} \mid \mathbf{a}^\top \mathbf{x} \leq b\}$
- 多面体: $\{\mathbf{x} \mid \mathbf{A}\mathbf{x} \preceq \mathbf{b}\}$
- 椭球体
- 范数锥: $\{(\mathbf{x}, t) \mid \|\mathbf{x}\| \leq t\}$

保凸运算:

- 任意交集 $\bigcap_{\alpha} C_{\alpha}$
- 仿射映射 $f(C)$
- 透视函数 / 线性分式函数

凸函数 (Convex Functions)

函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 为凸函数, 若其定义域是凸集且对任意 $\mathbf{x}, \mathbf{y}$ 和 $\theta \in [0, 1]$:

$$f(\theta \mathbf{x} + (1 - \theta) \mathbf{y}) \leq \theta f(\mathbf{x}) + (1 - \theta) f(\mathbf{y})$$

判别条件:

- 一阶条件 (可微时): $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$
- 二阶条件 (二阶可微时): $\nabla^2 f(\mathbf{x}) \succcurlyeq \mathbf{0}$ (Hessian 半正定)
- 上镜图 (Epigraph): $\text{epi}(f) = \{(\mathbf{x}, t) \mid f(\mathbf{x}) \leq t\}$ 是凸集 $\iff f$ 是凸函数

强凸函数: $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{\mu}{2} \|\mathbf{y} - \mathbf{x}\|^2$, $\mu > 0$ 唯一极小点。

凸优化问题

标准凸优化问题:

$$\begin{array}{ll}\min_{\mathbf{x}} & f_0(\mathbf{x}) \\ \text{s.t.} & f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & \mathbf{Ax} = \mathbf{b}\end{array}$$

其中 $f_0, f_1, \dots, f_m$ 均为凸函数。

关键性质

任何局部最优解 = 全局最优解。KKT 条件既是必要也是充分的。

常见凸优化问题:

- LP, QP (二次规划), QCQP (二次约束二次规划)
- SOCP (二阶锥规划), SDP (半定规划)
- 几何规划 (Geometric Programming)

凸对偶理论

Lagrange 对偶函数:

$$g(\boldsymbol{\lambda}, \boldsymbol{\nu}) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu})$$

其中 $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f_0(\mathbf{x}) + \sum_i \lambda_i f_i(\mathbf{x}) + \sum_j \nu_j (\mathbf{a}_j^\top \mathbf{x} - b_j)$ 。

Slater 条件 (Slater's Condition)

若存在严格可行点 (所有不等式约束严格成立), 则强对偶成立:

$$\text{Primal 最优值} = \text{Dual 最优值}$$

对偶问题始终是凹的 (极大化), 且自动给出原始问题的下界。

次梯度方法 (Subgradient Method)

对于不可微的凸函数 f :

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k, \quad \mathbf{g}_k \in \partial f(\mathbf{x}_k)$$

其中 $\partial f(\mathbf{x}) =$ 次梯度集合。

- 步长满足 $\alpha_k \rightarrow 0, \sum \alpha_k = \infty$ 时收敛
- 收敛速度 $\mathcal{O}(1/\sqrt{k})$
- ℓ_1 正则化、SVM 等场景常用

注意

次梯度不一定下降方向，需跟踪迄今最佳点（而非最后一步）。

近端梯度法 (Proximal Gradient Method)

目标: $\min f(\mathbf{x}) + g(\mathbf{x})$, f 光滑凸, g 可能不可微但 prox 易算。

近端算子 (Proximal Operator):

$$\text{prox}_{\eta g}(\mathbf{v}) = \arg \min_{\mathbf{x}} \left\{ g(\mathbf{x}) + \frac{1}{2\eta} \|\mathbf{x} - \mathbf{v}\|^2 \right\}$$

ISTA / FISTA (迭代 / 快速收缩阈值算法):

$$\mathbf{z}_k = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k)$$

$$\mathbf{x}_{k+1} = \text{prox}_{\eta g}(\mathbf{z}_k)$$

- 适用于 ℓ_1 正则化 (LASSO)、低秩矩阵恢复等
- FISTA 加速至 $\mathcal{O}(1/k^2)$

Nesterov 加速梯度 (Accelerated Gradient)

对于光滑凸函数，经典梯度下降 $\mathcal{O}(1/k)$ ，而 Nesterov 加速可达 $\mathcal{O}(1/k^2)$ 。

Nesterov 加速梯度 (NAG):

$$\begin{aligned}\mathbf{y}_k &= \mathbf{x}_k + \beta_k(\mathbf{x}_k - \mathbf{x}_{k-1}) \\ \mathbf{x}_{k+1} &= \mathbf{y}_k - \alpha_k \nabla f(\mathbf{y}_k)\end{aligned}$$

其中 $\beta_k = \frac{k-1}{k+2}$ (经典方案)。

- "Look ahead" 策略：不在当前位置求梯度，而在外推位置
- 对强凸函数可达线性收敛速率 $\mathcal{O}((1 - \sqrt{\mu/L})^k)$
- Nesterov 动量是 SGD with Momentum 和 Adam 的前身

ADMM (Alternating Direction Method of Multipliers)

适用于结构分离的凸优化:

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{z}} \quad & f(\mathbf{x}) + g(\mathbf{z}) \\ \text{s.t.} \quad & \mathbf{Ax} + \mathbf{Bz} = \mathbf{c} \end{aligned}$$

增广 Lagrange: $\mathcal{L}_\rho(\mathbf{x}, \mathbf{z}, \mathbf{y}) = f(\mathbf{x}) + g(\mathbf{z}) + \mathbf{y}^\top (\mathbf{Ax} + \mathbf{Bz} - \mathbf{c}) + \frac{\rho}{2} \|\mathbf{Ax} + \mathbf{Bz} - \mathbf{c}\|^2$

ADMM 迭代:

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \mathcal{L}_\rho(\mathbf{x}, \mathbf{z}_k, \mathbf{y}_k)$$

$$\mathbf{z}_{k+1} = \arg \min_{\mathbf{z}} \mathcal{L}_\rho(\mathbf{x}_{k+1}, \mathbf{z}, \mathbf{y}_k)$$

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \rho(\mathbf{Ax}_{k+1} + \mathbf{Bz}_{k+1} - \mathbf{c})$$

分布式优化、图像处理、统计学习等领域广泛应用。

半定规划 (SDP) 简介

SDP 是 LP 向矩阵空间的推广:

$$\begin{aligned} \min_{\mathbf{X}} \quad & \langle \mathbf{C}, \mathbf{X} \rangle = \text{tr}(\mathbf{C}^\top \mathbf{X}) \\ \text{s.t.} \quad & \langle \mathbf{A}_i, \mathbf{X} \rangle = b_i, \quad i = 1, \dots, m \\ & \mathbf{X} \succcurlyeq \mathbf{0} \quad (\text{半正定约束}) \end{aligned}$$

- 内点法 (Primal-Dual IPM) 可在多项式时间内求解
- 应用: 矩阵补全、鲁棒 PCA、组合优化的 SDP 松弛
- Max-Cut 的 Goemans–Williamson 0.878 近似比即基于 SDP

LP $\subset$ SOCP $\subset$ SDP, 逐层增加表达能力。

凸优化求解器概览

通用求解器:

- CVXPY (Python)
- CVX (MATLAB)
- Convex.jl (Julia)

专用求解器:

- ECOS, SCS —SOCP, SDP
- OSQP —QP
- MOSEK —商业级 LP/QP/SOCP/SDP
- Gurobi / CPLEX —LP, MILP, QP

建模原则:

- 若能建模为 LP $\rightarrow$ 用 LP 求解器 (最快)
- 若能建模为 QP $\rightarrow$ 用 QP 求解器
- 若能建模为 SOCP/SDP $\rightarrow$ 用相应锥规划求解器
- 只有非凸时才考虑通用 NLP 求解器 (IPOPT, SNOPT 等)

机器学习 = 优化

机器学习的核心是优化经验风险：

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{x}_i, y_i; \theta) + \lambda R(\theta)$$

- $\ell(\cdot)$: 损失函数 (交叉熵、MSE、Hinge 等)
- $R(\cdot)$: 正则化项 (ℓ_2 , ℓ_1 , dropout 等)
- N : 样本量 (现代 ML 中常为百万 十亿级)

ML 优化的特点：

- N 极大 $\rightarrow$ 随机梯度方法
- n (参数维度) 极大 $\rightarrow$ 一阶方法为主
- 非凸但实践中可找到不错的局部极小点
- 对泛化误差的关心 $\gg$ 对训练误差的关心

随机梯度下降 (SGD)

每次用一个小批量 (mini-batch) $\mathcal{B}$ 估计梯度:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \cdot \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}_i, y_i; \boldsymbol{\theta}_t)$$

- 无偏估计: $\mathbb{E}[\nabla f_{\mathcal{B}}(\boldsymbol{\theta})] = \nabla f(\boldsymbol{\theta})$
- 收敛速度 $\mathcal{O}(1/\sqrt{T})$ (非凸), $\mathcal{O}(1/T)$ (强凸)
- 关键技巧: 学习率调度、batch size 选择

瓶颈

纯 SGD 收敛慢, 对 ill-conditioned 和梯度噪声敏感。

动量法 (Momentum) 与 Nesterov

SGD with Momentum (Polyak):

$$\begin{aligned}\mathbf{v}_t &= \beta \mathbf{v}_{t-1} + \eta \nabla f(\boldsymbol{\theta}_t) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \mathbf{v}_t\end{aligned}$$

Nesterov Accelerated Gradient (NAG):

$$\begin{aligned}\mathbf{v}_t &= \beta \mathbf{v}_{t-1} + \eta \nabla f(\boldsymbol{\theta}_t - \beta \mathbf{v}_{t-1}) \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \mathbf{v}_t\end{aligned}$$

- 动量累积历史梯度方向，加速收敛、减少震荡
- NAG 先根据动量外推，再计算梯度 ("Look-ahead")
- 典型 $\beta = 0.9$

Adam (Adaptive Moment Estimation)

结合动量与自适应学习率：

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

- $\hat{\mathbf{m}}_t$ ：一阶动量（指数移动平均）
- $\hat{\mathbf{v}}_t$ ：二阶动量（RMS 缩放，类似对角的二阶方法近似）
- 默认参数： $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- 变体：AdamW（解耦 weight decay）、AdaBelief、LAMB

学习率调度 (Learning Rate Schedule)

学习率 η 对收敛至关重要：

- **Step decay**: 每隔固定 epoch 乘以衰减因子
- **Cosine annealing**:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{t\pi}{T}\right)$$

- **Warmup**: 前几个 epoch 线性增大 η (Transformer 训练关键)
- **Cyclic / OneCycle**: 周期性调整 LR, 加速收敛

Warmup 的必要性

Transformer 训练初期, Warmup 稳定训练、防止梯度爆炸; 没有 warmup 时 Adam 在初始阶段方差过大。

二阶方法与深度学习

深度学习主要用一阶方法，但二阶方法有吸引力：

- **Natural Gradient / K-FAC**：用 Fisher 信息矩阵近似 Hessian
- **Shampoo**：逐层预处理，Google 大规模训练使用
- **AdaHessian**：用 Hutchinson 方法随机估计 Hessian 对角线
- **Newton Sketch**：随机投影加速 Newton 法

挑战：Hessian 维度 $n \times n$ 太大（数十亿参数 $\rightarrow$ 不可能存储），且深度网络 Hessian 谱复杂（负特征值、大条件数）。

分布式训练中的优化

大模型 ($> 100B$ 参数) 需要跨数千 GPU 的分布式训练。

数据并行 (Data Parallel) + All-Reduce:

- 每 GPU 持有完整模型副本, 处理不同数据 mini-batch
- 求完梯度后 All-Reduce 求平均、更新

模型并行 (Model Parallel):

- Tensor Parallelism: 矩阵乘法在多个 GPU 上分片
- Pipeline Parallelism: 不同层在不同 GPU 上

关键优化技术:

- **Gradient Accumulation:** 多步累积梯度后更新 (模拟大 batch)
- **混合精度 (FP16/BF16):** 降低通信和计算开销
- **ZeRO (Zero Redundancy Optimizer):** 分片优化器状态

ZeRO 优化器 (DeepSpeed)

Microsoft DeepSpeed 的 ZeRO 将优化器状态分区以节省 GPU 显存:

阶段	分片内容	显存节省
ZeRO-1	优化器状态 (Adam m, v)	4×
ZeRO-2	优化器状态 + 梯度	8×
ZeRO-3	优化器状态 + 梯度 + 参数	线性随 GPU 数扩展

这使 175B 参数的 GPT-3 级模型可以在数百个 GPU 上训练，而不需要数千个。

LLM 微调：优化视角

全参数微调 (Full Fine-tuning) 对千亿参数模型代价极高：

- 需要存储所有参数梯度
- 需要 16-32 倍模型参数量的显存（优化器状态 + 梯度 + 激活值）

参数高效微调 (PEFT) 大幅减少可训练参数：

- **LoRA**：低秩矩阵分解，仅训练低秩增量 $\Delta W = BA$
- **Adapter**：在层间插入小型瓶颈网络
- **Prefix / Prompt Tuning**：仅优化虚拟 token 嵌入

本质上都是降维优化——约束优化在一个低维子流形上进行。

RLHF 中的优化

RLHF (Reinforcement Learning from Human Feedback) 的优化流程:

- ① SFT (Supervised Fine-Tuning): 标准监督学习
- ② 训练 Reward Model: Bradley-Terry 偏好模型
- ③ PPO (Proximal Policy Optimization): 用 Reward Model 优化策略

$$\max_{\theta} \mathbb{E} [r(x, y) - \beta \cdot \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})]$$

替代方案: **DPO (Direct Preference Optimization)**——将 RL 优化转化为一个直接的分类损失, 避免了显式的 reward 建模和 PPO 训练。

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

大模型推理部署的优化技术：

- **权重量化 (GPTQ, AWQ)**：将 FP16 权重压缩到 4-bit 或 8-bit
 - 基于最优脑外科 (OBS) 的逐层量化
 - 目标： $\min_{\hat{W}} \|WX - \hat{W}X\|_F^2$
- **KV Cache 优化**：量化缓存的 Key-Value 对
- **投机解码 (Speculative Decoding)**：用小模型生成候选 token，大模型验证
- **FlashAttention**：IO-aware 的精确注意力计算

这些都是经典的近似优化问题：在精度、速度、内存之间寻找 Pareto 最优。

大模型训练中的优化挑战

LLM 训练的优化难点：

- **损失尖峰 (Loss Spike)**: 训练不稳定，需要回滚和重新开始
- **数值精度**: FP16/BF16 下的梯度 underflow / overflow
- **梯度裁剪 (Gradient Clipping)**: $\|g\| > \tau$ 时缩放 $g \leftarrow \tau \cdot g / \|g\|$
- **Z-loss / Logit Regularization**: 防止 logit 漂移
- **嵌入层归一化**: output embedding 的 LayerNorm 稳定训练

Chinchilla Scaling Law: 在计算预算 C 固定时，模型参数 N 与训练 token 数 D 的最优配置：

$$N_{\text{opt}} \propto C^{0.5}, \quad D_{\text{opt}} \propto C^{0.5}$$

运筹优化与 ML 的交叉前沿

- **Decision-Focused Learning**: 将 ML 预测嵌入优化模型, 端到端训练
 - 例: 预测需求 $\rightarrow$ 库存优化, 梯度需穿过 LP/IP 层
- **Learning to Optimize**: 用 RL 学习分支定界启发式、切平面选择等
- **组合优化的神经求解器**: Pointer Network / Transformer 解 TSP, VRP
- **混合整数规划 + ML**: 用 GNN 加速 MILP 求解
- **Convex ML**: 凸损失 + 凸正则项的模型 (如 Elastic Net)
- **Bilevel Optimization**: 元学习、超参数优化、对抗训练

为什么 GPU 对优化算法至关重要？

现代 ML 优化的核心计算是大规模矩阵运算和梯度计算——天然适合 GPU 的 SIMT 架构。

CPU (少核大算力):

- 8–64 个复杂核心
- 擅长分支、串行逻辑
- 大缓存、乱序执行

GPU (多核轻量):

- 数千个简单核心
- 擅长数据并行 (SIMT)
- 高内存带宽 (~ 2 TB/s)

NVIDIA CUDA 编程模型

- Thread $\rightarrow$ Warp (32 threads) $\rightarrow$ Block $\rightarrow$ Grid
- 内存层次: Global $\rightarrow$ L2 $\rightarrow$ Shared Memory $\rightarrow$ Registers
- Tensor Cores (Volta+): 单周期 4×4 矩阵乘加

A100 80GB: 6912 CUDA cores + 432 Tensor Cores, 312 TFLOPS (FP16)

GPU 上的优化算法：一阶方法的天然优势

梯度下降族算法 (SGD, Adam, AdamW) 的每次迭代：

- ① Forward pass: 矩阵乘法 $y = Wx$
- ② Loss 计算: 标量, 计算量小
- ③ Backward pass: 矩阵乘法 $\nabla_W \mathcal{L} = (\nabla_y \mathcal{L}) x^\top$
- ④ 参数更新: 逐元素操作

关键观察：第 1、3 步是密集矩阵乘法，GPU 加速 $50\times-200\times$ 。

- cuBLAS: 高度优化的 GEMM (通用矩阵乘)
- cuDNN: 卷积、池化、归一化的 GPU kernel
- 二阶方法 (Newton, K-FAC) 的 Hessian 构造在 GPU 上仍昂贵

GPU 瓶颈

不是计算，是**显存带宽和通信**。一个 175B 模型需要 ~ 350 GB 仅存参数 (FP16)，加上优化器状态轻松破 TB。

混合精度训练 (Mixed Precision)

动机： FP32 精度在很多梯度更新中冗余，FP16 可翻倍吞吐、减半显存。

标准混合精度流程 (NVIDIA APEX / PyTorch AMP)：

- ① 权重以 FP32 主副本存储
- ② Forward/Backward 用 FP16 计算 (速度 $2\times$)
- ③ **Loss Scaling**：梯度太小时乘以常数 S 防止 underflow
- ④ 梯度回转为 FP32，更新主副本权重

BF16 (Brain Float 16)

- 与 FP32 相同指数位 (8 bit)，仅尾数截断 (7 bit)
- 动态范围 = FP32，无需 loss scaling
- NVIDIA A100 / H100 原生支持，训练更稳定

Tensor Cores 在 FP16/BF16 下吞吐可达 FP32 的 $8-16\times$ 。

梯度累积 & 大批量训练

GPU 显存限制单次能处理的 batch size，但大批量训练有助于收敛稳定性和扩展性。

梯度累积 (Gradient Accumulation):

- 执行 k 次 micro-batch 前向 + 反向，梯度累加但不更新
- 第 k 次后统一更新，等价于 $\text{batch size} \times k$
- 显存需求 = micro-batch size，训练时间 $\approx$ 不变（梯度同步开销略增）

大批量训练的挑战:

- 线性缩放规则 (Linear Scaling Rule, Goyal et al. 2017):
 $\eta_{\text{new}} = \eta_{\text{base}} \times (B_{\text{new}}/B_{\text{base}})$ ，需配合 warmup
- LARS (Layer-wise Adaptive Rate Scaling): 逐层自适应学习率
- LAMB: Adam 的大批量变体，BERT 预训练标配

分布式 GPU 训练架构

数据并行 (Data Parallel):

- 每 GPU 持有完整模型
- 各 GPU 吃不同 mini-batch
- All-Reduce 求梯度平均
- PyTorch DDP, Horovod

模型并行:

- Tensor Parallel (Megatron-LM)
- Pipeline Parallel (GPipe)
- 3D Parallel = DP + TP + PP

NCCL 通信原语:

- All-Reduce: 梯度平均
- All-Gather: 收集分片
- Reduce-Scatter: 分片归约
- 带宽: ~ 600 GB/s (NVLink)

FSDP (Fully Sharded DP):

参数 + 梯度 + 优化器全分片, ZeRO-3 的 PyTorch 原生实现。

显存优化策略

训练大模型的显存构成 (以 1B 参数 + Adam + FP16 为例):

组成部分	估算大小	比例
模型参数 (FP16)	2 GB	~20%
Adam m (FP32)	4 GB	~40%
Adam v (FP32)	4 GB	~40%
梯度 (FP16)	2 GB	~20%
激活值 (Activations)	可变	额外

核心优化技术:

- **Gradient Checkpointing**: 不存中间激活, 反向时重新计算 (显存 $\searrow$ 、计算 $\nearrow$)
- **Activation Offloading**: 激活值溢出到 CPU 内存
- **FlashAttention**: IO-aware 精确注意力, 分块计算 + 在线 softmax, 显存从 $\mathcal{O}(N^2)$ 降到 $\mathcal{O}(N)$
- **Kernel Fusion**: 融合多个 CUDA kernel 减少访存 (如 Bias + GeLU + Dropout)

GPU 优化实战 Checklist

- ① **混合精度**: 启用 AMP (FP16) 或 BF16, 确认 Tensor Core 利用率
- ② **数据加载**: 多进程 DataLoader, prefetch, pin_memory
- ③ **梯度累积**: 用小 batch + 累积模拟大 batch
- ④ **分布式**: 单机多卡用 DDP, 多机用 FSDP / DeepSpeed
- ⑤ **编译优化**: torch.compile / TensorRT, XLA, kernel fusion
- ⑥ **显存管理**: gradient checkpointing, FlashAttention, fused optimizer
- ⑦ **监控工具**: nvidia-smi, Nsight Systems (timeline), Nsight Compute (kernel)
- ⑧ **batch size 调优**: 找到 GPU 利用率拐点 (通常 > 80% 为佳)

核心原则: 减少 GPU 空闲等待——要么在算, 要么在传数据, 不要闲下来。

GPU 加速的运筹优化应用

- **大规模 LP 求解**: GPU 加速内点法的线性系统求解 (cuSOLVER)
- **分支定界并行**: MIP 的 B&B 树在 GPU 上并行展开
- **组合优化的神经求解器**: Transformer/GNN 在 GPU 上做 TSP/VRP 端到端推理
- **蒙特卡洛仿真**: GPU 加速随机优化 (如随机规划的场景生成与求解)
- **RL for OR**: GPU 并行仿真环境, PPO 训练调度/路径策略
- **SDP 求解**: GPU 加速 SDP 内点法中的 Cholesky 分解和矩阵乘法

趋势

NVIDIA cuOpt: 专为运筹优化设计的 GPU 加速库, 支持 VRP、调度等问题, 速度可达传统求解器的 100 $\times$ 。

总结

- ① 线性规划是优化理论的基石，单纯形法与内点法互补
- ② 非线性规划：KKT 条件统一处理约束问题，一阶/二阶方法各有适用场景
- ③ 凸优化：局部即全局，加速梯度、ADMM、近端方法解决大规模问题
- ④ ML 中的优化：SGD $\rightarrow$ Momentum $\rightarrow$ Adam，一阶方法主导
- ⑤ LLM 中的优化：分布式训练 (ZeRO)、PEFT (LoRA)、量化、RLHF
- ⑥ 交叉前沿：ML 与 OR 双向赋能，端到端优化决策
- ⑦ GPU 加速：CUDA、混合精度、分布式训练、FlashAttention、cuOpt

谢谢！